

Question: *There is a large object that is on the left side of the large blue cylinder in front of the rubber cylinder on the right side of the purple shiny thing; what is its shape?*
Effective Question: *What shape is a large object left of a cylinder?*

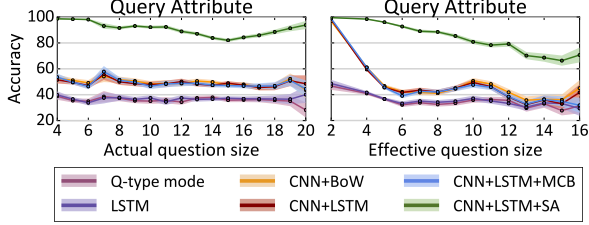
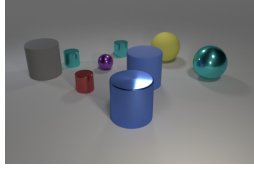


Figure 7. **Top:** Many questions can be answered correctly without correctly solving all subtasks. For a given question and scene we can prune functions from the question’s program to generate an *effective question* which is shorter but gives the same answer. **Bottom:** Accuracy on query questions vs. actual and effective question size. Accuracy decreases with effective question size but not with actual size. Shaded area shows a 95% confidence interval.

4.5. Effect of Question Size

Intuitively, longer questions should be harder since they involve more reasoning steps. We define a question’s *size* to be the number of functions in its program, and in Figure 7 (bottom left) we show accuracy on query-attribute questions as a function of question size.² Surprisingly accuracy appears unrelated to question size.

However, many questions can be correctly answered even when some subtasks are not solved correctly. For example, the question in Figure 7 (top) can be answered correctly without identifying the correct large blue cylinder, because all large objects left of a cylinder are cylinders.

To quantify this effect, we define the *effective question* of an image-question pair: we prune functions from the question’s program to find the smallest program that, when executed on the scene graph for the question’s image, gives the same answer as the original question.³ A question’s *effective size* is the size of its effective question. Questions whose effective size is smaller than their actual size need not be degenerate. The question in Figure 7 is not degenerate because the entire question is needed to resolve its object references (there are two blue cylinders and two rubber cylinders), but it has a small effective size since it can be correctly answered without resolving those references.

In Figure 7 (bottom), we show accuracy on query questions as a function of effective question size. The error rate of all models increases with effective question size, suggesting that models struggle with long reasoning chains.

²We exclude questions with same-attribute relations since their max size is 10, introducing unwanted correlations between size and difficulty. Excluded questions show the same trends (see supplementary material).

³Pruned questions may be *ill-posed* (Section 3) so they are executed with modified semantics; see supplementary material for details.

Question: *There is a purple cube that is in front of the yellow metal sphere; what material is it?*
Absolute question: *There is a purple cube in the front half of the image; what material is it?*

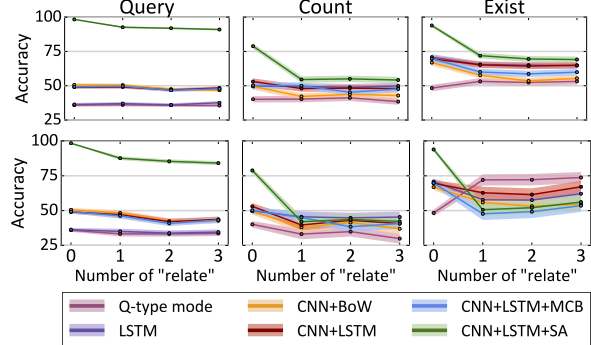
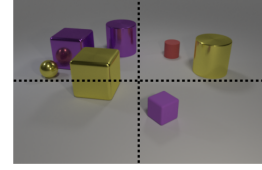


Figure 8. **Top:** Some questions can be correctly answered using *absolute* definitions for spatial relationships; for example in this image there is only one purple cube in the bottom half of the image. **Bottom:** Accuracy of each model on *chain-structured* questions as a function of the number of spatial relationships in the question, separated by question type. Top row shows all chain-structured questions; bottom row excludes questions that can be correctly answered using absolute spatial reasoning.

4.6. Spatial Reasoning

We expect that questions with more spatial relationships should be more challenging since they require longer chains of reasoning. The top set of plots in Figure 8 shows accuracy on chain-structured questions with different numbers of relationships.⁴ Across all three question types, CNN+LSTM+SA shows a significant drop in accuracy for questions with one or more spatial relationship; other models are largely unaffected by spatial relationships.

Spatial relationships force models to reason about objects’ relative positions. However, as shown in Figure 8, some questions can be answered using *absolute spatial reasoning*. In this question the purple cube can be found by simply looking in the bottom half of the image; reasoning about its position relative to the metal sphere is unnecessary.

Questions only requiring absolute spatial reasoning can be identified by modifying the semantics of spatial relationship functions in their programs: instead of returning sets of objects related to the input object, they ignore their input object and return the set of objects in the half of the image corresponding to the relationship. A question only requires absolute spatial reasoning if executing its program with these modified semantics does not change its answer.

The bottommost plots of Figure 8 show accuracy on chain-structured questions with different number of relationships, *excluding* questions that can be answered

⁴We restrict to chain-structured questions to avoid unwanted correlations between question topology and number of relationships.

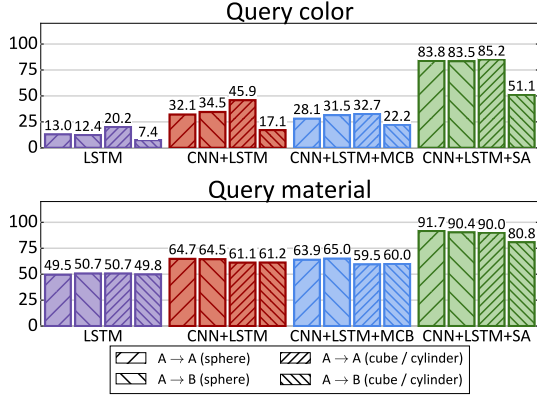


Figure 9. In *Condition A* all cubes are gray, blue, brown, or yellow and all cylinders are red, green, purple, or cyan; in *Condition B* color palettes are swapped. We train models in *Condition A* and test in both conditions to assess their generalization performance. We show accuracy on “query color” and “query material” questions, separating questions by shape of the object being queried.

with absolute spatial reasoning. On query questions, CNN+LSTM+SA performs significantly worse when absolute spatial reasoning is excluded; on count questions no model outperforms LSTM, and on exist questions no model outperforms Q-type mode. These results suggest that models have not learned the semantics of spatial relationships.

4.7. Compositional Generalization

Practical VQA systems should perform well on images and questions that contain novel combinations of attributes not seen during training. To do so models might need to learn disentangled representations for attributes, for example learning separate representations for color and shape instead of memorizing all possible color/shape combinations.

We can use CLEVR to test the ability of VQA models to perform such compositional generalization. We synthesize two new versions of CLEVR: in *Condition A* all cubes are gray, blue, brown, or yellow and all cylinders are red, green, purple, or cyan; in *Condition B* these shapes swap color palettes. Both conditions contain spheres of all eight colors.

We retrain models on *Condition A* and compare their performance when testing on *Condition A* ($A \rightarrow A$) and testing on *Condition B* ($A \rightarrow B$). In Figure 9 we show accuracy on query-color and query-material questions, separating questions asking about spheres (which are the same in *A* and *B*) and cubes/cylinders (which change from *A* to *B*).

Between $A \rightarrow A$ and $A \rightarrow B$, all models perform about the same when asked about the color of spheres, but perform much worse when asked about the color of cubes or cylinders; CNN+LSTM+SA drops from 85% to 51%. Models seem to learn strong biases about the colors of objects and cannot overcome these biases when conditions change.

When asked about the material of cubes and cylinders, CNN+LSTM+SA shows a smaller gap between $A \rightarrow A$ and

$A \rightarrow B$ (90% vs 81%); other models show no gap. Having seen metal cubes and red metal objects during training, models can understand the material of red metal cubes.

5. Discussion and Future Work

This paper has introduced CLEVR, a dataset designed to aid in diagnostic evaluation of visual question answering (VQA) systems by minimizing dataset bias and providing rich ground-truth representations for both images and questions. Our experiments demonstrate that CLEVR facilitates in-depth analysis not possible with other VQA datasets: our question representations allow us to slice the dataset along different axes (question type, relationship type, question topology, *etc.*), and comparing performance along these different axes allows us to better understand the reasoning capabilities of VQA systems. Our analysis has revealed several key shortcomings of current VQA systems:

- **Short-term memory:** All systems we tested performed poorly in situations requiring short-term memory, including attribute comparison and integer equality questions (Section 4.2), same-attribute relationships (Section 4.3), and tree-structured questions (Section 4.4). Attribute comparison questions are of particular interest, since models can successfully *identity* attributes of objects but struggle to *compare* attributes.
- **Long reasoning chains:** Systems struggle to answer questions requiring long chains of nontrivial reasoning, including questions with large effective sizes (Section 4.5) and count and existence questions with many spatial relationships (Section 4.6).
- **Spatial Relationships:** Models fail to learn the true semantics of spatial relationships, instead relying on absolute image position (Section 4.6).
- **Disentangled Representations:** By training and testing models on different data distributions (Section 4.7) we argue that models do not learn representations that properly disentangle object attributes; they seem to learn strong biases from the training data and cannot overcome these biases when conditions change.

Our study also shows cases where current VQA systems are successful. In particular, spatial attention [44] allows models to focus on objects and identify their attributes even on questions requiring multiple steps of reasoning.

These observations present clear avenues for future work on VQA. We plan to use CLEVR to study models with explicit short-term memory, facilitating comparisons between values [13, 18, 39, 42]; explore approaches that encourage learning disentangled representations [5]; and investigate methods that compile custom network architectures for different patterns of reasoning [2, 3]. We hope that diagnostic datasets like CLEVR will help guide future research in VQA and enable rapid progress on this important task.

Supplementary Material

A. Basic Functions

As described in Section 3 and shown in Figure 2, each question in CLEVR is associated with a functional program built from a set of basic functions. In this section we detail the semantics of these basic functional building blocks.

Data Types. Our basic functional building blocks operate on values of the following types:

- **Object:** A single object in the scene.
- **ObjectSet:** A set of zero or more objects in the scene.
- **Integer:** An integer between 0 and 10 (inclusive).
- **Boolean:** Either *yes* or *no*.
- **Value types:**
 - **Size:** One of *large* or *small*.
 - **Color:** One of *gray*, *red*, *blue*, *green*, *brown*, *purple*, *cyan*, or *yellow*.
 - **Shape:** One of *cube*, *sphere*, or *cylinder*.
 - **Material:** One of *rubber* or *metal*.
- **Relation:** One of *left*, *right*, *in front*, or *behind*.

Basic Functions. The functional program representations of questions are built from the following set of basic building blocks. Each of these functions takes the image’s scene graph as an additional implicit input.

- **scene** ($\emptyset \rightarrow \text{ObjectSet}$)
Returns the set of all objects in the scene.
- **unique** ($\text{ObjectSet} \rightarrow \text{Object}$)
If the input is a singleton set, then return it as a standalone *Object*; otherwise raise an exception and flag the question as *ill-posed* (See Section 3).
- **relate** ($\text{Object} \times \text{Relation} \rightarrow \text{ObjectSet}$)
Return all objects in the scene that have the specified spatial relation to the input object. For example if the input object is a red cube and the input relation is *left*, then return the set of all objects in the scene that are left of the red cube.
- **count** ($\text{ObjectSet} \rightarrow \text{Integer}$)
Returns the size of the input set.
- **exist** ($\text{ObjectSet} \rightarrow \text{Boolean}$)
Returns *yes* if the input set is nonempty and *no* if it is empty.
- **Filtering functions:** These functions filter the input objects by some attribute, returning the subset of input objects that match the input attribute. For example calling *filter_size* with the first input *small* will return the set of all small objects in the second input.

- **filter_size** ($\text{ObjectSet} \times \text{Size} \rightarrow \text{ObjectSet}$)
- **filter_color** ($\text{ObjectSet} \times \text{Color} \rightarrow \text{ObjectSet}$)
- **filter_material** ($\text{ObjectSet} \times \text{Material} \rightarrow \text{ObjectSet}$)
- **filter_shape** ($\text{ObjectSet} \times \text{Shape} \rightarrow \text{ObjectSet}$)
- **Query functions:** These functions return the specified attribute of the input object; for example calling *query_color* on a red object returns *red*.
 - **query_size** ($\text{Object} \rightarrow \text{Size}$)
 - **query_color** ($\text{Object} \rightarrow \text{Color}$)
 - **query_material** ($\text{Object} \rightarrow \text{Material}$)
 - **query_shape** ($\text{Object} \rightarrow \text{Shape}$)
- **Logical operators:**
 - **AND** ($\text{ObjectSet} \times \text{ObjectSet} \rightarrow \text{ObjectSet}$)
Returns the intersection of the two input sets.
 - **OR** ($\text{ObjectSet} \times \text{ObjectSet} \rightarrow \text{ObjectSet}$)
Returns the union of the two input sets.
- **Same-attribute relations:** These functions return the set of objects that have the same attribute value as the input object, not including the input object. For example calling *same_shape* on a cube returns the set of all cubes in the scene, excluding the query cube.
 - **same_size** ($\text{Object} \rightarrow \text{ObjectSet}$)
 - **same_color** ($\text{Object} \rightarrow \text{ObjectSet}$)
 - **same_material** ($\text{Object} \rightarrow \text{ObjectSet}$)
 - **same_shape** ($\text{Object} \rightarrow \text{ObjectSet}$)
- **Integer comparison:** Checks whether the two integer inputs are equal, or whether the first is less than or greater than the second, returning either *yes* or *no*.
 - **equal_integer** ($\text{Integer} \times \text{Integer} \rightarrow \text{Boolean}$)
 - **less_than** ($\text{Integer} \times \text{Integer} \rightarrow \text{Boolean}$)
 - **greater_than** ($\text{Integer} \times \text{Integer} \rightarrow \text{Boolean}$)
- **Attribute comparison:** These functions return *yes* if their inputs are equal and *no* if they are not equal.
 - **equal_size** ($\text{Size} \times \text{Size} \rightarrow \text{Boolean}$)
 - **equal_material** ($\text{Material} \times \text{Material} \rightarrow \text{Boolean}$)
 - **equal_color** ($\text{Color} \times \text{Color} \rightarrow \text{Boolean}$)
 - **equal_shape** ($\text{Shape} \times \text{Shape} \rightarrow \text{Boolean}$)

B. Effective Question Size

In Section 4.5 we note that some questions can be correctly answered without correctly resolving all intermediate object references, and define a question’s *effective question* to quantitatively measure this effect.

For any question we can compute its *effective question* by pruning functions from the question’s program; the effective question is the smallest such pruned program that, when executed on the scene graph for the question’s image, gives the same answer as the original question.